COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Oct 4, 2026	**DATE SUBMITTED:**	Oct 6, 2026
NAME:	Serrano, Gabriel Immanuel	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 10.1 Graphs

1. PROCEDURE OUTPUT

A.1. Create a Graph

Graphs > iloa.cpp > ...

```cpp
#include <iostream>

const int MAX = 20;

class Graph{
    char vertices[MAX];
    char edgesFrom[MAX][MAX];
    char edgesTo[MAX][MAX];
    int verticesCount;
    int edgesCount;

    bool exists(char v) {
        for (int i = 0; i < verticesCount; i++)
            if (vertices[i] == v) return true;
        return false;
    }
public:
    Graph() {
        verticesCount = 0;
        edgesCount = 0;
    }

    void addVertex(char v) {
        if (exists(v)) {
            std::cout << "Vertex " << v << " already exists" << std::endl;
            return;
        }
        if (verticesCount == MAX) {
            std::cout << "Graph is full" << std::endl;
            return;
        }
        vertices[verticesCount] = v;
        verticesCount++;
    }
```

```
35
36      void addEdge(char a, char b) {
37          if (!exists(a) || !exists(b)) {
38              std::cout << "Vertex not found" << std::endl;
39              return;
40          }
41          edgesFrom[edgesCount][0] = a;
42          edgesTo[edgesCount][0] = b;
43          edgesCount++;
44      }
45
46      void displayVertices() {
47          std::cout << "Vertices: ";
48          for (int i = 0; i < verticesCount; i++)
49              std::cout << vertices[i] << " ";
50          std::cout << std::endl;
51      }
52
53      void displayEdges() {
54          std::cout << "Edges: ";
55          for (int i = 0; i < edgesCount; i++)
56              std::cout << "(" << edgesFrom[i][0] << ", " << edgesTo[i][0] << ") ";
57          std::cout << std::endl;
58      }
59  };
```

```
61  int main(){
62      Graph g;
63      g.addVertex('A');
64      g.addVertex('B');
65      g.addVertex('C');
66      g.addVertex('D');
67      g.addVertex('E');
68      g.addEdge('A', 'B');
69      g.addEdge('A', 'C');
70      g.addEdge('A', 'D');
71      g.addEdge('B', 'E');
72      g.addEdge('C', 'D');
73      g.addEdge('D', 'E');
74      g.displayVertices();
75      g.displayEdges();
76      return 0;
77  }
```

```
Vertices: A B C D E
Edges: (A, B) (A, C) (A, D) (B, E) (C, D) (D, E)
```

Observation :
Based on this code, we created a Graph class that stores up to 20 vertices and their edges. The addVertex() function is used to add the vertices A, B, C, D, and E, while the addEdge() function connects the vertices together. The code then uses displayVertices() and displayEdges() to display all the vertices and the connections between them.

A.2. Adjacency Matrix

Graphs > iloa2.cpp > ...

```cpp
#include <iostream>

const int Max = 20;

class GraphMatrix{
    char vertices[Max];
    int matrix[Max][Max];
    int verticesCount;
    bool directed;

    int indexOf(char v){
        for(int i = 0; i < verticesCount; i++)
            if(vertices[i] == v) return i;
        return -1;
    }

public:
    GraphMatrix(bool isDirected){
        verticesCount = 0;
        directed = isDirected;
        for(int i = 0; i < Max; i++)
            for(int j = 0; j < Max; j++)
                matrix[i][j] = 0;
    }

    void addVertex(char v){
        if(indexOf(v) != -1 || verticesCount == Max) return;
        vertices[verticesCount] = v;
        verticesCount++;
    }

    void addEdge(char a, char b, int weight = 1){
        int i = indexOf(a);
        int j = indexOf(b);
        if(i == -1 || j == -1) return;
        matrix[i][j] = weight;
        if(!directed) matrix[j][i] = weight;
    }
```

```
39
40      void displayVertices(){
41          std::cout << "Vertices: ";
42          for(int i = 0; i < verticesCount; i++)
43              std::cout << vertices[i] << " ";
44          std::cout << std::endl;
45      }
46
47      void displayMatrix(){
48          std::cout << " ";
49          for(int i = 0; i < verticesCount; i++)
50              std::cout << " " << vertices[i];
51          std::cout << std::endl;
52          for(int i = 0; i < verticesCount; i++){
53              std::cout << vertices[i];
54              for(int j = 0; j < verticesCount; j++)
55                  std::cout << " " << matrix[i][j];
56              std::cout << std::endl;
57          }
58      }
59  };
```

```
63  int main() {
64      std::cout << "Undirected Graph" << std::endl;
65      GraphMatrix g1(false);
66      g1.addVertex('A');
67      g1.addVertex('B');
68      g1.addVertex('C');
69      g1.addVertex('D');
70      g1.addVertex('E');
71      g1.addEdge('A', 'B');
72      g1.addEdge('A', 'C');
73      g1.addEdge('A', 'D');
74      g1.addEdge('B', 'E');
75      g1.addEdge('C', 'D');
76      g1.addEdge('D', 'E');
77      g1.displayVertices();
78      g1.displayMatrix();
```

```
80      std::cout << std::endl << "Weighted Graph" << std::endl;
81      GraphMatrix g2(true);
82      g2.addVertex('A');
83      g2.addVertex('B');
84      g2.addVertex('C');
85      g2.addVertex('D');
86      g2.addVertex('E');
87      g2.addEdge('A', 'B', 4);
88      g2.addEdge('A', 'C', 3);
89      g2.addEdge('C', 'D', 2);
90      g2.addEdge('D', 'B', 1);
91      g2.addEdge('D', 'E', 1);
92      g2.addEdge('E', 'A', 1);
93      g2.displayVertices();
94      g2.displayMatrix();
95      return 0;
96  }
```

```
Undirected Graph
Vertices: A B C D E
  A B C D E
A 0 1 1 1 0
B 1 0 0 0 1
C 1 0 0 1 0
D 1 0 1 0 1
E 0 1 0 1 0

Weighted Graph
Vertices: A B C D E
  A B C D E
A 0 4 3 0 0
B 0 0 0 0 0
C 0 0 0 2 0
D 0 1 0 0 1
E 1 0 0 0 0
```

Observation : Based on this code, we created a GraphMatrix class that uses an adjacent matrix to store the connections between the vertices. The code first creates an undirected graph with vertices A to E, where the connections are stored as value of 1 in the matrix. Then, it creates a weighted directed graph version where different numbers are used to represent the weights of the edges. Lastly the displayVertices and displayMatrix functions are used to show the vertices and their corresponding matrix.

A.3. Adjacency List

Graphs > C sc1.h > ...

```h
#ifndef SC1_H
#define SC1_H

#include <iostream>

struct adjNode{
    int val, cost;
    adjNode* next;
};

struct graphEdge{
    int start_ver, end_ver, weight;
};

class DiaGraph{
    adjNode* getAdjListNode(int value, int weight, adjNode* head){
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head;
        return newNode;
    }
    int N;
public:
    adjNode **head;
    //constructor
    DiaGraph(graphEdge edges[], int n, int N){
        head = new adjNode*[N]();
        this->N = N;

        for(int i = 0; i < N; i++){
            head[i] = nullptr;
        }

        for(unsigned i = 0; i < n; i++){
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            head[start_ver] = newNode;
        }
    }
```

```
45
46      //Destructor
47      ~DiaGraph(){
48          for(int i = 0; i < N; i++)
49              delete[] head[i];
50          delete[] head;
51      }
52  };
53
54  void display_AdjList(adjNode* ptr, int i){
55      while(ptr != nullptr){
56          std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
57          ptr = ptr->next;
58      }
59      std::cout << std::endl;
60  }
61
62
63  #endif
```

Graphs > sc1.cpp > main()

```
1  #include "sc1.h"
2  #include <iostream>
3
4  int main(){
5
6      graphEdge edges[] = {{0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}};
7      int N = 6;
8      int n = sizeof(edges)/sizeof(edges[0]);
9      DiaGraph diagraph(edges, n, N);
10     std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;
11     for (int i = 0; i < N; i++){
12         display_AdjList(diagraph.head[i], i);
13     }
14     return 0;
15 }
```

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Observation : Based on this code, we created a directed graph using an adjacency list to store the vertices, edges, and their weights. The graphEdge array stores the starting vertex, end vertex, and the weight of each edge. While the DiaGraph class creates the adjacency list for the graph which is then displayed by the display_Adjlist function that goes through each vertex and displays the connected vertices and edge weights.

B.1. Depth-First Search

Graphs > sc2.cpp > ...

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N){
    }

    auto vertices() const{
        return V;
    }

    auto &edges() const{
        return edge_list;
    }

    void add_edge(Edge<T> &&e){
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const{
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

  private:
      size_t V;
      std::vector<Edge<T>> edge_list;
  };
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```cpp
template <typename T>
void test_DFS()
{
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main(){
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

Observation: Based on this code, we created a graph that stores vertices and their connected edges using a vector and map. The create_reference_graph() functions creates the graph and adds the connections between the vertices, while the depth_first_search function uses a stack implementation and a set to perform the DFS traversal. The code starts traversing at vertex 1 and visits the connected vertices while making sure that each vertex is only visited once.

B.2. Breadth-First Search

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const{
        return V;
    }

    auto &edges() const{
        return edge_list;
    }

    void add_edge(Edge<T> &&e){
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```
61      auto outgoing_edges(size_t v) const{
62          std::vector<Edge<T>> edges_from_v;
63          for (auto &e : edge_list)
64          {
65              if (e.src == v)
66              {
67                  edges_from_v.emplace_back(e);
68              }
69          }
70          return edges_from_v;
71      }
```

```
73      template <typename U>
74      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
75
76  private:
77      size_t V;
78      std::vector<Edge<T>> edge_list;
79  };
80
81  template <typename T>
82  auto create_reference_graph(){
83      Graph<T> G(9);
84
85      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
86      edges[1] = {{2, 2}, {5, 3}};
87      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
88      edges[3] = {{4, 2}, {7, 3}};
89      edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
90      edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
91      edges[6] = {{4, 4}, {7, 4}, {8, 1}};
92      edges[7] = {{3, 3}, {6, 4}};
93      edges[8] = {{4, 5}, {5, 3}, {6, 1}};
94
95      for (auto &i : edges)
96          for (auto &j : i.second)
97              G.add_edge(Edge<T>{i.first, j.first, j.second});
98
99      return G;
100 }
```

```cpp

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest){
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1);
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

```cpp

template <typename T>
void test_BFS(){
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main(){
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Observation : Based on this code, we created a weighted graph where each vertex is connected to other vertices with a specific edge weight. The BFS search function uses a queue implementation instead of a stack to process the vertices in the order they were discovered, allowing the program to check the graph level by level. Starting from vertex 1, the program adds its connected vertices to the queue and continues processing the next vertices while the visited set prevents the same vertex from being processed repeatedly.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

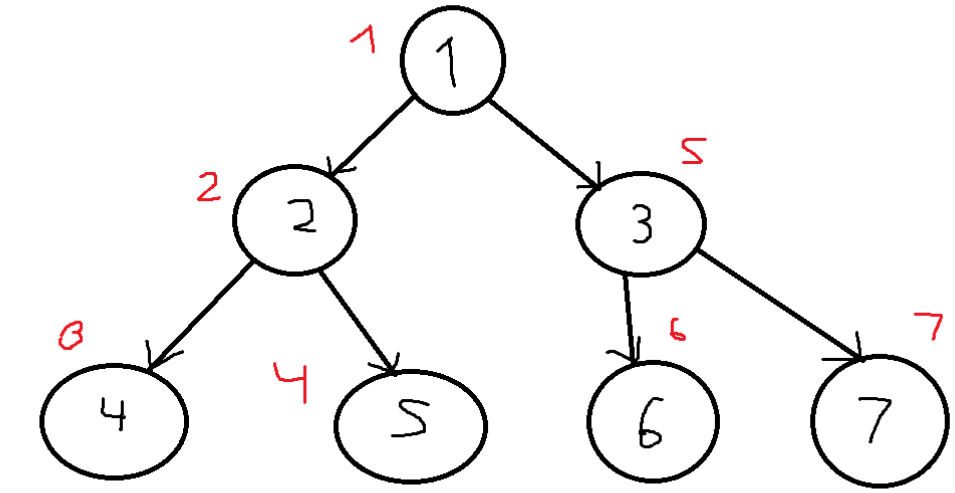
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- Depth-First Search (DFS) would be the most helpful algorithm. It visits one vertex and continues along a path as deeply as possible before backtracking and exploring another path. This matches the situation because the person wants to finish exploring one path before going back and trying another vertex.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- The equivalent of DFS in tree traversal is Depth-First Tree Traversal, which includes Preorder. It goes deep into a branch before going back and exploring another branch. In the example diagram you can see it goes to the 1 2 4 5 3 6 7, showing how it explores the deeper vertices first before moving to the other branches.




```cpp
#include <iostream>

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
};

TreeNode* newNode(int value) {
    TreeNode* node = new TreeNode();
    node->val = value;
    node->left = nullptr;
    node->right = nullptr;
    return node;
}

void preorder(TreeNode* node){
    if (node == nullptr) return;
    std::cout << node->val << " ";
    preorder(node->left);
    preorder(node->right);
}
```

```
23
24
25  int main(){
26      TreeNode* root = newNode(1);
27      root->left = newNode(2);
28      root->right = newNode(3);
29      root->left->left = newNode(4);
30      root->left->right = newNode(5);
31      root->right->left = newNode(6);
32      root->right->right = newNode(7);
33
34      std::cout << "Preorder:  ";
35      preorder(root);
36      std::cout << std::endl;
37
38      return 0;
39  }
```

```
Preorder:  1 2 4 5 3 6 7
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

- The data structure used for the BFS is a queue implementation. Using a queue implementation allows us to store the vertices that need to be visited and it should follow the FIFO order. In the code shown above , std::queue<size_t> was used to store the vertices during traversal. This allows the BFS to visit the current level of the vertices before moving to the next level.

4. How many times can a node be visited in the BFS?

- In the BFS code, each node is visited only once. The visited set is used to check whether a node has already been visited before adding or processing it. Once a node is marked as visited, it will not be processed again. This prevents the traversal from repeatedly visiting the same node, even if there are multiple connections to it

Conclusion :

I learned how to implement and understand how graphs work by using adjacency lists and adjacency matrices. Through this activity, I became familiar with different types of graphs, like directed, undirected, and weighted graphs, as well as how the vertices and edges are connected. I also learned how DFS and BFS works and how they use different data structures, like a stack implementation for DFS and a queue implementation for BFS. The most challenging part for me was understanding how the traversal order will work because it was difficult to follow which vertex should be visited next, especially when there are many connections between the vertices. Overall, this activity helped me improve my knowledge and foundation in implementing graph algorithms.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, Code assistance and Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.